MIDAP: A Full-Stack Open NPU Platform for Embedded AI Exploration

openmidap.github.io

Changjae Yi, Hyunsu Moh, Seongwoo Choi, Joon Choi, Youngchul Yoon, Soonhoi Ha, CAP Lab, Seoul National University

Keywords: Open-source NPU, HW/SW co-design, MLIR compiler, Cycle-accurate virtual prototyping

University Demonstration · DAC 2026 · Long Beach, CA

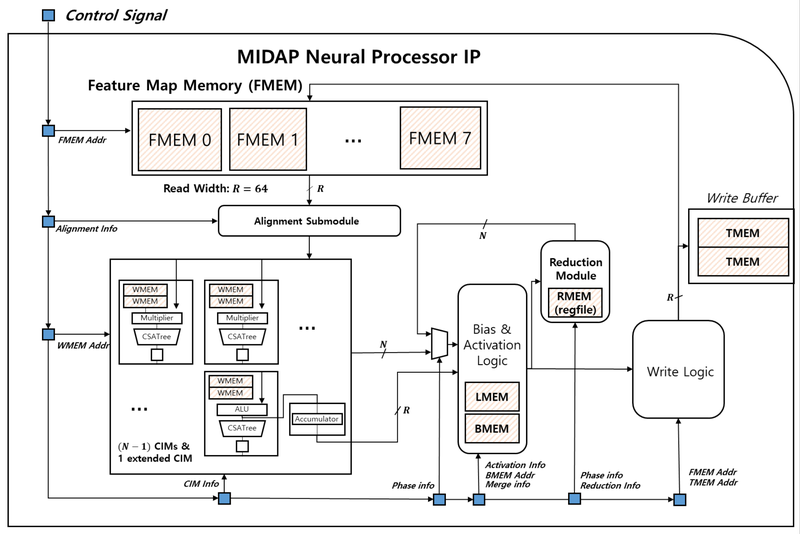
Opening the NPU Stack

Many NPU stacks are black boxes: compiler decisions are invisible, architectures are frozen, and every "what if" costs an RTL respin. RISC-V broke that cycle for CPUs by giving everyone an open baseline to build on. MIDAP is that baseline for embedded NPUs with the complete toolchain that drives it. Every intermediate artifact (IRs, schedules, traces, microcode, waveforms) stays open to inspection, and the whole stack goes open source in H1 2027.

The NPU: MIDAP

Architecture

- Adder-tree-based PE
  - CIM (convolution-in-memory) [1]
- Decoupled on-chip memory
  → weight and feature access never contend
- High on-chip memory reuse
  → low DRAM traffic across layer boundaries
- Contention-free micro/macro-pipelining
  → no stalls between fused layers



MIDAP architecture

① The Entire Network, On One Accelerator

16+ operators with **no** CPU/GPU offloading

- High MAC utilization
- Low DRAM access

Convolution	GEMM / GEMV / FC	Activation · GELU LUT	Dilated Convolution
Depthwise Convolution	Element-wise Add/Mult	Transposed Convolution	Pooling (max · avg)
Global Pooling	Nearest Neighbor Upsampling	Bilinear Upsampling	Implicit Transpose

■ native datapath [1] ■ extended CIM [7]
■ reduction module [7] ■ tensor virtualization [2] ■ compiler technique [3, 4]

② One Accelerator for Diverse AI Models

Supporting CNNs (class., det., seg.) and Transformers

Classification	MobileNetV1/V2/V3, ResNet-50/101/152, WideResNet, SE-ResNet, EfficientNets (B0, B1, …), Inception-v3, etc.
Object Detection	YOLOv3/v4/v4-Tiny/v5/v8, MobileNet-SSD, etc.
Semantic Segmentation	DeepLabv3+, U-Net
Generative Models	DCGAN, DiscoGAN
Vision Transformers	ViT-Base [3]
Language Models	Gemma1/2 [4]

③ Performance of Silicon-Ready

1 Extended PE, 1024 MACs, 534MHz (**1.093 TOPS**)

70.6%
MAC utilization, YOLOv4-tiny

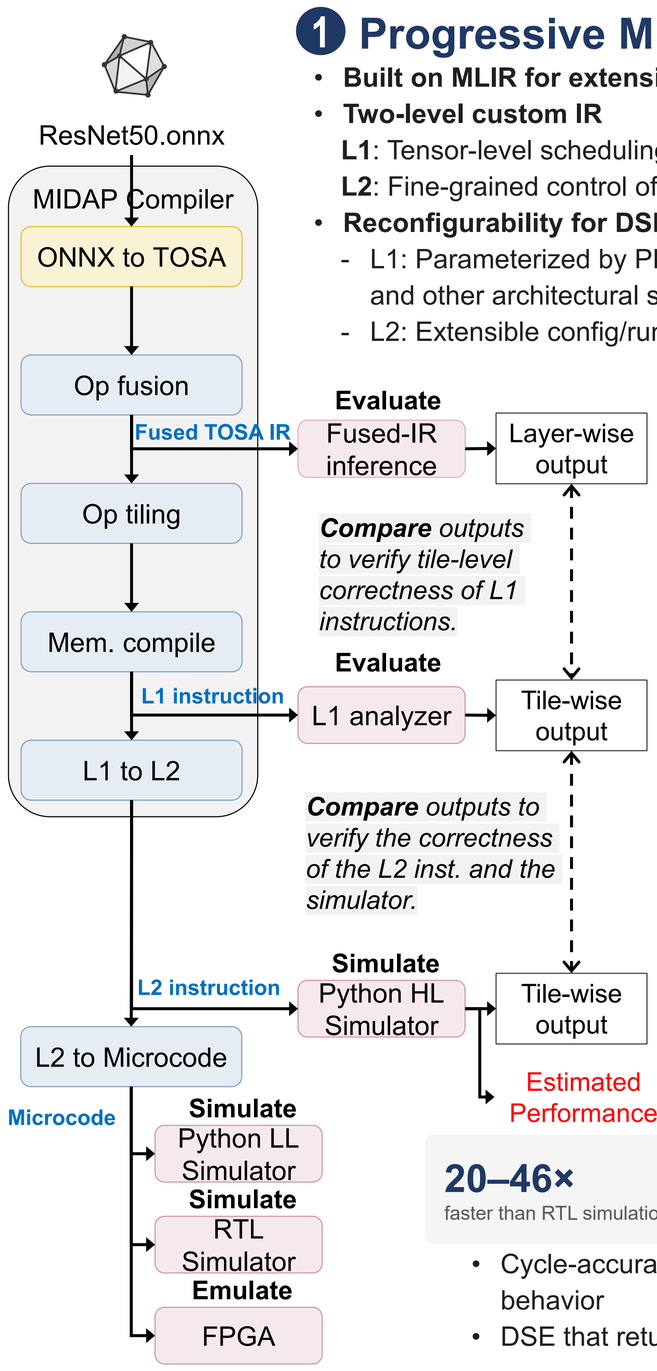
1.83 TOPS/W
DeepLabV3+

Performance, Power & Efficiency (ASIC, post-layout + SDF)

Workload	Latency	MAC Util.	Power	TOPS/W
YOLOv4-tiny	12.66ms	70.57%	0.425 W	1.81
DeepLabV3+	27.79ms	67.8%	0.404 W	1.83
EfficientNet-b2	11.13ms	13.83%	0.225 W	0.67

Samsung 14 nm, tapeout-ready: 1.093 TOPS @ 534 MHz · 2.608 mm² (66.7% of area: SRAM).

The Full-Stack Platform



① Progressive MLIR lowering

- **Built on MLIR for extensibility and ecosystem reuse**
- **Two-level custom IR**
  L1: Tensor-level scheduling with low-level control abstracted away
  L2: Fine-grained control of PEs and DMAs
- **Reconfigurability for DSE & customization**
  - L1: Parameterized by PE-array dimensions, on-chip SRAM capacity, and other architectural specifications
  - L2: Extensible config/run/wait instruction structure

Compilation passes

- **ONNX to TOSA**
  Utilize a minimal yet expressive op set with precise numerical semantics
- **TOSA to L1**

Pass	What it does
Op Fusion	maps a sequence of ops onto the PE pipeline and fuses them into an L1 layer operation
Op Tiling	splits each layer into subtasks that fit within PE and on-chip SRAM constraints
Memory Compile	performs static memory allocation and schedule optimization

- **L1 to L2**
  Generate config / run / wait operations from L1 layer & memory operations

② Virtual Prototyping

Predict perf. before you build

20–46×
faster than RTL simulation [5]

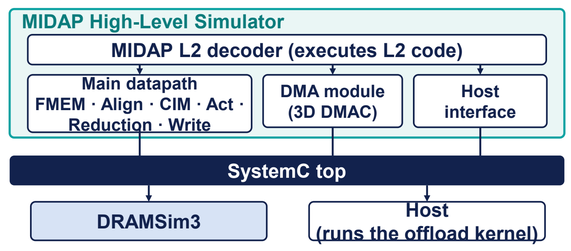
≤ 6.8%
end-to-end latency error vs RTL [5]

- Cycle-accurate virtual prototype models DRAM and DMA behavior
- DSE that returns clear design verdicts

③ Modular by design: explore it, then extend it

Tunable components in the virtual prototype

(W, N) parallelism	SRAM (WMEM / FMEM) size	Datapath extensions (e.g. SIMD)
Clock frequency	DRAM type & bandwidth	DMA controller model



④ Where MIDAP stands

NVDLA	Open RTL and reference software stack, functional VP, no longer actively developed
Gemmini	Parameterized HW generator, external SW runtimes, cycle accuracy via RTL sim
Google Coral NPU	Deployment-oriented open IP, MLIR toolchain, cycle accuracy via RTL sim
MIDAP	**Modular, extensible co-design stack. Compiler, cycle-accurate virtual prototype, and RTL testbench in one observable loop**

On the Roadmap

- **More Models & optimizations in the compiler**
- **Toolchain support for Transformer models**
- **2027 H1 Open Release**

- Architecture
- Demo videos
- Publications
- Waitlist

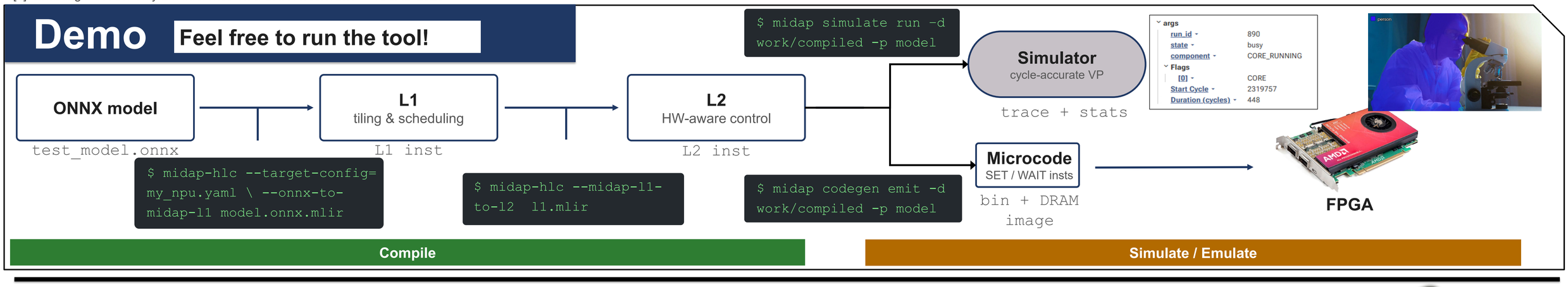
openmidap.github.io

References

[1] A Novel CNN Accelerator Enabling Fully-Pipelined Execution of Layers — ICCD 2019
[2] Tensor Virtualization Technique to Support Efficient Data Reorganization for CNN Accelerators — DAC 2020
[3] Vision Transformer Inference on a CNN Accelerator — ICCD 2024
[4] Enabling Decoder-only LM Inference on a CNN Accelerator — ICCAD 2025
[5] Fast & Accurate NPU Virtual Prototyping w/ Analytical Memory Modeling — RSP 2023
[6] Ph.D. thesis (D. Kang), Seoul National University
[7] Datapath Extension of NPUs to Support Nonconvolutional Layers Efficiently — IEEE Design & Test

Demo

Feel free to run the tool!



ENGINEERING
COLLEGE OF ENGINEERING
SEOUL NATIONAL UNIVERSITY
서 울 대 학 교 공 과 대 학

LABORATORY